

CLOVE Restaurant - Deployment Guide

Comprehensive guide for deploying the CLOVE AI-powered restaurant ordering system to production environments.

Table of Contents

1. [Prerequisites](#)
2. [Local Development Setup](#)
3. [Docker Deployment](#)
4. [Cloud Deployment](#)
5. [Environment Configuration](#)
6. [Database Setup](#)
7. [Security Best Practices](#)
8. [Monitoring and Logging](#)
9. [Troubleshooting](#)
10. [Scaling](#)

Prerequisites

- Python 3.8 or higher
- Docker and Docker Compose (for containerized deployment)
- Git for version control
- Access to Anthropic Claude API key
- (Optional) Cloud provider account (AWS, GCP, Azure, Heroku)

Local Development Setup

Step 1: Clone Repository

```
git clone https://github.com/yourusername/clove-restaurant.git
cd clove-restaurant
```

Step 2: Create Python Virtual Environment

```
python -m venv venv
source venv/bin/activate # On Windows: venv\Scripts\activate
```

Step 3: Install Dependencies

```
pip install -r requirements.txt
```

Step 4: Configure Environment Variables

Create a `.env` file in the project root:

```
ANTHROPIC_API_KEY=sk-ant-xxxxxxxxxxxx
OPENAI_API_KEY=sk-proj-xxxxxxxxxxxx # Optional
```

```
ADMIN_USERNAME=admin
ADMIN_PASSWORD=your-secure-password-here
DB_PATH=./src/database/menu.db
```

Step 5: Initialize Database

```
python src/database/setup_db.py
```

This creates the SQLite database and initializes tables with sample menu data.

Step 6: Run Development Server

```
uvicorn server:app --reload --host 0.0.0.0 --port 8000
```

Access the application at:

- Chatbot: <http://localhost:8000>
- Admin: <http://localhost:8000/admin>
- API Docs: <http://localhost:8000/docs>

Docker Deployment

Step 1: Create Dockerfile

Create `Dockerfile` in project root:

```
FROM python:3.11-slim

WORKDIR /app

# Install system dependencies
RUN apt-get update && apt-get install -y \
    sqlite3 \
    && rm -rf /var/lib/apt/lists/*

# Copy requirements and install Python dependencies
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy application code
COPY . .

# Initialize database if it doesn't exist
RUN python src/database/setup_db.py || true

# Expose port
EXPOSE 8000

# Run uvicorn server
```

```
CMD ["uvicorn", "server:app", "--host", "0.0.0.0", "--port", "8000", "--workers", "4"]
```

Step 2: Create Docker Compose File

Create `docker-compose.yml` :

```
version: '3.8'

services:
  web:
    build: .
    ports:
      - "8000:8000"
    environment:
      ANTHROPIC_API_KEY: ${ANTHROPIC_API_KEY}
      OPENAI_API_KEY: ${OPENAI_API_KEY}
      ADMIN_USERNAME: ${ADMIN_USERNAME}
      ADMIN_PASSWORD: ${ADMIN_PASSWORD}
      DB_PATH: ./src/database/menu.db
    volumes:
      - ./src/database:/app/src/database
      - ./logs:/app/logs
    restart: unless-stopped
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8000/docs"]
      interval: 30s
      timeout: 10s
      retries: 3
      start_period: 10s
```

Step 3: Build and Run with Docker

```
# Build the image
docker build -t clove-restaurant:latest .

# Run the container
docker run -d \
  -p 8000:8000 \
  -e ANTHROPIC_API_KEY=your_api_key \
  -e ADMIN_USERNAME=admin \
  -e ADMIN_PASSWORD=secure_password \
  -v $(pwd)/src/database:/app/src/database \
  -v $(pwd)/logs:/app/logs \
  --name clove-app \
  clove-restaurant:latest

# Or use Docker Compose
docker-compose up -d
```

Cloud Deployment

AWS EC2

1. Launch EC2 instance (Ubuntu 22.04 LTS)
2. Install Docker:

```
sudo apt-get update
sudo apt-get install -y docker.io
sudo usermod -aG docker $USER
```

3. Clone repository and deploy using Docker Compose
4. Configure security groups to allow HTTP (80) and HTTPS (443)
5. Set up Nginx reverse proxy (see below)

AWS Elastic Beanstalk

1. Install EB CLI:

```
pip install awsebcli
```

2. Initialize EB application:

```
eb init -p python-3.11 clove-restaurant
```

3. Create environment:

```
eb create production
```

4. Set environment variables:

```
eb setenv ANTHROPIC_API_KEY=your_key ADMIN_USERNAME=admin
ADMIN_PASSWORD=secure_password
```

5. Deploy:

```
eb deploy
```

Heroku

1. Install Heroku CLI and login:

```
brew install heroku/brew/heroku
heroku login
```

2. Create Procfile:

```
web: uvicorn server:app --host 0.0.0.0 --port $PORT --workers 4
```

3. Create app and deploy:

```
heroku create clove-restaurant
heroku config:set ANTHROPIC_API_KEY=your_key
heroku config:set ADMIN_USERNAME=admin
heroku config:set ADMIN_PASSWORD=secure_password
git push heroku main
```

4. View logs:

```
heroku logs --tail
```

Google Cloud Run

1. Create Dockerfile (as above)

2. Build and push to Container Registry:

```
gcloud builds submit --tag gcr.io/your-project/clove-restaurant
```

3. Deploy to Cloud Run:

```
gcloud run deploy clove-restaurant \
  --image gcr.io/your-project/clove-restaurant \
  --platform managed \
  --region us-central1 \
  --set-env-vars
  ANTHROPIC_API_KEY=your_key,ADMIN_USERNAME=admin,ADMIN_PASSWORD=secure_password
```

Azure Container Instances

1. Create Azure Container Registry

2. Build and push image:

```
az acr build --registry your-registry --image clove-restaurant:latest .
```

3. Deploy:

```
az container create \
  --resource-group your-group \
  --name clove-app \
  --image your-registry.azurecr.io/clove-restaurant:latest \
  --environment-variables ANTHROPIC_API_KEY=your_key
```

Environment Configuration

Required Variables

Variable	Description	Default	Example
ANTHROPIC_API_KEY	Claude API key (required)	None	sk-ant-...
ADMIN_USERNAME	Admin login username	None	admin
ADMIN_PASSWORD	Admin login password	None	secure_password
DB_PATH	SQLite database path	./src/database/menu.db	/data/menu.db

Optional Variables

Variable	Description	Default
OPENAI_API_KEY	OpenAI API key	None
CORS_ORIGINS	Allowed CORS origins	["*"]
SESSION_TIMEOUT	Session timeout in hours	8

Production Configuration Example

```
ANTHROPIC_API_KEY=sk-ant-xxxxxxxxxxxxx
ADMIN_USERNAME=admin
ADMIN_PASSWORD=very-secure-password-with-special-chars!@#%$
DB_PATH=/var/lib/clove/menu.db
CORS_ORIGINS=["https://yourdomain.com"]
```

Database Setup

Automatic Initialization

The database is automatically initialized when running:

```
python src/database/setup_db.py
```

This creates:

- categories table with menu categories
- menu_items table with restaurant items
- orders table for order tracking

Manual Database Backup

```
# Backup database
cp src/database/menu.db src/database/menu.db.backup

# Restore from backup
cp src/database/menu.db.backup src/database/menu.db
```

Database Migrations

For production, consider using SQLAlchemy with Alembic for schema migrations:

```
pip install sqlalchemy alembic
alembic init migrations
alembic revision --autogenerate -m "description"
alembic upgrade head
```

Security Best Practices

1. API Key Management

- Never commit `.env` files
- Use environment variables in production
- Rotate API keys periodically
- Use separate keys for development and production

2. Password Security

Passwords are hashed using PBKDF2-SHA256 with 260k iterations:

```
from passlib.context import CryptContext

pwd_context = CryptContext(schemes=["pbkdf2_sha256"], deprecated="auto")
hashed_password = pwd_context.hash("password")
```

3. HTTPS/SSL Setup

Using Nginx as Reverse Proxy

Create `/etc/nginx/sites-available/clove` :

```
server {
    listen 80;
    server_name yourdomain.com;

    # Redirect HTTP to HTTPS
    return 301 https://$server_name$request_uri;
}

server {
    listen 443 ssl http2;
    server_name yourdomain.com;

    # SSL certificates (use Let's Encrypt)
    ssl_certificate /etc/letsencrypt/live/yourdomain.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/yourdomain.com/privkey.pem;

    # Security headers
    add_header Strict-Transport-Security "max-age=31536000" always;
    add_header X-Frame-Options "DENY" always;
```

```

add_header X-Content-Type-Options "nosniff" always;

location / {
    proxy_pass http://localhost:8000;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}
}

```

Enable site:

```

sudo ln -s /etc/nginx/sites-available/clove /etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl restart nginx

```

4. Session Management

- Sessions expire after 8 hours
- Cookies are HTTP-only and SameSite=Lax
- Use secure cookies in production (HTTPS only)

5. Firewall Rules

```

# Allow only necessary ports
sudo ufw allow 22/tcp      # SSH
sudo ufw allow 80/tcp      # HTTP
sudo ufw allow 443/tcp     # HTTPS
sudo ufw default deny incoming
sudo ufw enable

```

6. Rate Limiting

Add rate limiting to FastAPI:

```

from slowapi import Limiter
from slowapi.util import get_remote_address

limiter = Limiter(key_func=get_remote_address)
app.state.limiter = limiter

@app.post("/api/chat")
@limiter.limit("10/minute")
async def chat_endpoint(request: Request, body: dict):
    ...

```

Monitoring and Logging

Application Logging

Logs are written to `./logs/` directory. Configure logging in `server.py` :

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s',
    handlers=[
        logging.FileHandler('logs/app.log'),
        logging.StreamHandler()
    ]
)
```

Log Rotation

Use logrotate on Linux:

Create `/etc/logrotate.d/clove` :

```
/app/logs/*.log {
    daily
    rotate 7
    compress
    delaycompress
    notifempty
    create 0640 appuser appuser
}
```

Health Checks

Add health check endpoint:

```
@app.get("/health")
async def health():
    return {"status": "healthy"}
```

Use in monitoring:

```
curl http://localhost:8000/health
```

Application Monitoring

Using Prometheus

```
from prometheus_client import Counter, Histogram, generate_latest

request_count = Counter('http_requests_total', 'Total HTTP requests')
request_latency = Histogram('http_request_duration_seconds', 'HTTP request latency')
```

```
@app.middleware("http")
async def add_prometheus_metrics(request: Request, call_next):
    request_count.inc()
    with request_latency.time():
        response = await call_next(request)
    return response

@app.get("/metrics")
async def metrics():
    return generate_latest()
```

Using Sentry for Error Tracking

```
import sentry_sdk
from sentry_sdk.integrations.fastapi import FastApiIntegration

sentry_sdk.init(
    dsn="your-sentry-dsn",
    integrations=[FastApiIntegration()],
    traces_sample_rate=1.0,
    environment="production"
)
```

Troubleshooting

Issue: Database Not Found

```
# Solution: Initialize database
python src/database/setup_db.py
```

Issue: API Key Not Working

```
# Check if .env file exists
cat .env

# Verify API key is correct
echo $ANTHROPIC_API_KEY
```

Issue: Port Already in Use

```
# Find process using port 8000
lsof -i :8000

# Kill process
kill -9 <PID>
```

```
# Or use different port
unicorn server:app --port 8001
```

Issue: Docker Container Exits

```
# Check logs
docker logs <container-id>

# Run interactively to see errors
docker run -it clove-restaurant:latest
```

Issue: Database Locked Error

```
# Remove lock file
rm -f src/database/menu.db-journal

# Restart application
```

Issue: High Memory Usage

```
# Reduce worker count
unicorn server:app --workers 2

# Monitor memory
docker stats <container-id>
```

Scaling

Horizontal Scaling with Load Balancer

Use Nginx upstream to distribute traffic:

```
upstream clove_backend {
    server localhost:8001;
    server localhost:8002;
    server localhost:8003;
    server localhost:8004;
}

server {
    listen 80;
    server_name yourdomain.com;

    location / {
        proxy_pass http://clove_backend;
    }
}
```

Run multiple instances:

```
for i in {1..4}; do
  docker run -d -p 8000$i:8000 clove-restaurant:latest
done
```

Database Scaling

For SQLite, consider migrating to PostgreSQL for production:

1. Install PostgreSQL driver: `pip install psycopg2-binary`
2. Update database connection string
3. Run migrations to new database
4. Update connection pooling

Container Orchestration

Kubernetes Deployment

Create `k8s-deployment.yaml` :

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: clove-restaurant
spec:
  replicas: 3
  selector:
    matchLabels:
      app: clove
  template:
    metadata:
      labels:
        app: clove
    spec:
      containers:
        - name: clove
          image: clove-restaurant:latest
          ports:
            - containerPort: 8000
          env:
            - name: ANTHROPIC_API_KEY
              valueFrom:
                secretKeyRef:
                  name: clove-secrets
                  key: api-key
      resources:
        requests:
          memory: "256Mi"
          cpu: "250m"
        limits:
          memory: "512Mi"
```

```
    cpu: "500m"
  livenessProbe:
    httpGet:
      path: /health
      port: 8000
    initialDelaySeconds: 10
    periodSeconds: 10
```

Deploy:

```
kubectl apply -f k8s-deployment.yaml
```

Maintenance

Regular Tasks

- Daily: Monitor logs and error rates
- Weekly: Backup database
- Monthly: Update dependencies
- Quarterly: Security audit and penetration testing

Updating Dependencies

```
# Check for updates
pip list --outdated

# Update specific package
pip install --upgrade fastapi

# Update all
pip install --upgrade -r requirements.txt
pip freeze > requirements.txt
```

Support and Documentation

For more information, see:

- Main README: `README.md`
- FastAPI Documentation: <https://fastapi.tiangolo.com/>
- Anthropic Claude Documentation: <https://docs.anthropic.com/>
- Docker Documentation: <https://docs.docker.com/>

Rollback Procedure

In case of issues after deployment:

```
# Docker rollback
docker run -d --name clove-rollback clove-restaurant:previous-tag

# Git rollback
```

```
git revert <commit-hash>  
git push
```

```
# Database rollback
```

```
cp src/database/menu.db.backup src/database/menu.db
```